

Smart MCQ Solver

Multi-Model Approaches to Five-Option Multiple-Choice Question Answering

Course: BSDA2001P — Deep Learning and Generative AI | Kaggle: Smart MCQ Solver Challenge

Krishna | 23f2001849 | WandB Project: 23f2001849-t22026

Abstract. This report presents a multi-model pipeline for the Smart MCQ Solver Challenge, a five-option multiple-choice question answering task scored with MAP@3. Three models were built and tracked with full Weights & Biases logging: a TF-IDF baseline, a TextCNN trained end to end with no pretrained weights, and a fine-tuned bert-base-uncased transformer. The submitted pipeline reached **0.75561 MAP@3** on the public leaderboard, against 0.352 for a constant prediction. A key early finding, 257 exact-duplicate rows shared between train and test, shaped how every validation number in this project was interpreted, and several negative results are reported alongside what worked.

1 Introduction

Problem statement. Given a question prompt and five labelled options (A–E), predict the correct option. The competition provides 2,000 labelled training questions and 500 unlabelled test questions, spanning a wide mix of topics (physics, philosophy, general academic trivia) with no single dominant domain. Scoring uses **MAP@3**: a model submits its top-3 guesses per question, ranked by confidence, and receives $1/(\text{rank} + 1)$ credit if the correct answer is not ranked first, so a model does not need to be certain, it needs its top-3 ranking to be good.

Why this is non-trivial. Distractor options are frequently near-paraphrases of the correct answer, differing by a single technical term. With only 2,000 training rows spread across many topics, every model here has limited examples per topic to learn from, which shaped both the choice of architectures and the decision to compare several rather than rely on one.

1.1 Dataset Characteristics

Both files are complete, with no missing values. Prompts average ~ 118 characters and options ~ 165 , and the correct answer is mildly imbalanced across options, with B and C together accounting for $\sim 47\%$ of answers against $\sim 16\%$ for E alone. Correct options also run about 21 characters longer than distractors on average.

EDA finding: exact-duplicate training rows

Six prefix phrases (“Pick the best possible answer:”) and six suffix phrases (“among the listed options”) recur across the prompts, none of them carrying information about the answer. Stripping both reduces train from **1,758 unique prompts to 252 distinct questions** and test from 492 to 228, and 44 of those 252 carry more than one correct-answer text across their paraphrases. Separately, **257 of the 500 test rows are exact duplicates of a training row** (prompt and all five options), and every test question already appears in train in some form. Both facts matter for how the validation scores in Section 4 should be read.

2 Methodology

No cleaning or lowercasing was applied: the corpus is machine-generated and well punctuated, and a single digit can be the whole answer. The one transform is an expansion to *pairwise ranking* format: each question becomes five rows, one per option, carrying a binary label, so train goes from 2,000 rows to 10,000 and test from 500 to 2,500. A model scores each row and the three highest-scoring options become the prediction. The same expanded table feeds all three models, so the comparison is between architectures rather than input representations. Cross-validation is 5-fold GroupKFold on the question id throughout.

2.1 Model 1 — TF-IDF + Logistic Regression (baseline)

- **Features.** Word-level TF-IDF (uni- to trigrams, 80,000 features, sublinear term frequency) over the prompt–option pair, plus character-level TF-IDF (char_wb, 2–5 grams, 50,000 features) over the divergent tail of the option, stacked with three overlap features and classified with logistic regression.
- **Role and limitation.** Trains in minutes on CPU and sets the floor every other model must clear. It has no word order, no synonymy and no embedding space, so two options differing by one token are separated only by the character n -grams over that token.

2.2 Model 2 — TextCNN, trained from scratch

- **Architecture.** Following Kim (2014), with a single-logit ranking head. A randomly initialised 64-dimensional embedding feeds three parallel Conv1d layers of width 3, 4 and 5 with 64 filters each, then ReLU, global max-pooling over time, concatenation to 192 dimensions, dropout 0.5 and a linear layer to one score.
- **Trained from scratch.** Every weight, from the embedding table to the head, is learned purely from the 2,000 training questions. No pretrained backbone or external vocabulary is used anywhere in this model.
- **Why it suits the task.** A width- k convolution along the token axis is a learned k -gram detector, and global max-pooling makes detection position-invariant, which suits data where the distinguishing clause of an option sits at an arbitrary offset.

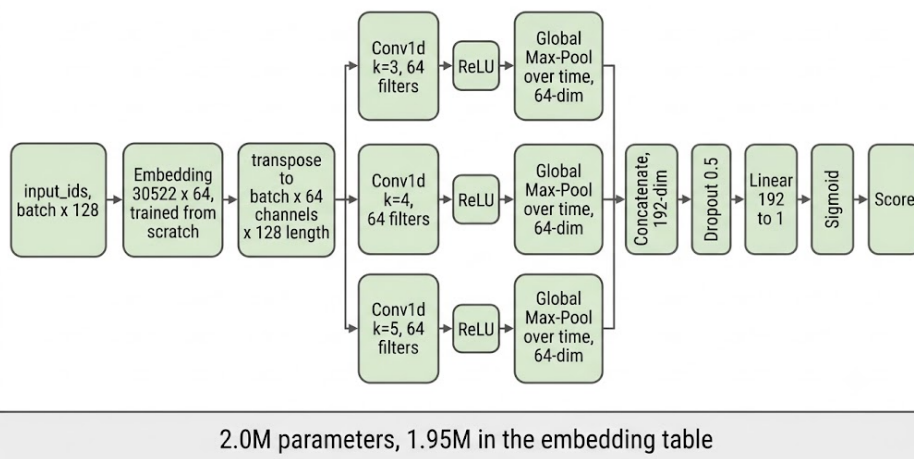


Figure 1: TextCNN architecture: embedding → three parallel convolution widths → global max-pooling → concatenation → dropout → one relevance score per (question, option) pair.

2.3 Model 3 — bert-base-uncased (fine-tuned)

The encoder half of the Transformer of Vaswani et al. (2017): 12 layers, each a 12-head self-attention block followed by a feed-forward network of inner size 3,072, hidden size 768, 109.5M parameters. Hugging Face’s `AutoModelForSequenceClassification` with `num_labels=1` puts one linear layer on the pooled `[CLS]` token, so each (prompt, option) pair is scored independently as a binary correct/incorrect problem rather than as a 5-way softmax.

Neither earlier model can know what a redshift is, so all world knowledge must arrive with the checkpoint. Self-attention also lets prompt and option form one sequence, so every option token attends directly to every prompt token rather than being compared through surface overlap.

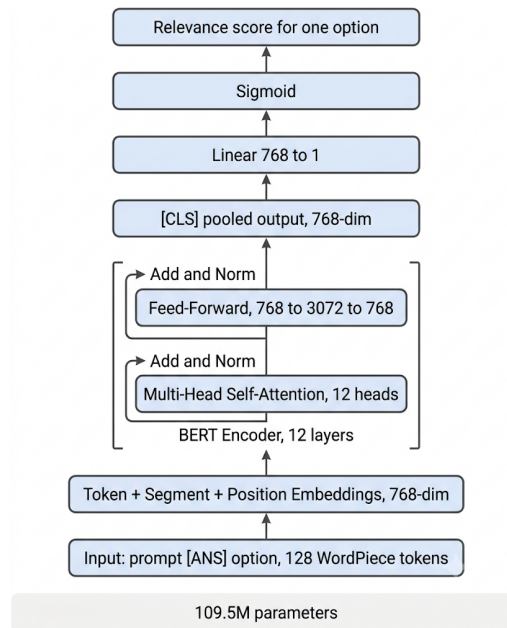


Figure 2: BERT ranker: prompt and option are encoded as one 128-token sequence and the pooled `[CLS]` vector is projected to a single relevance score, run once per option.

2.4 Tokenisation Strategy

- **Neural branch.** WordPiece paired with `bert-base-uncased`, a 30,522-token vocabulary, with `max_length` 128, truncation and padding through Hugging Face’s `AutoTokenizer`. It was chosen over a scratch tokeniser because the corpus is far too small to learn a vocabulary, and a scratch one would fragment the scientific terms that carry the answer. The identical tensors feed TextCNN and BERT, so Section 4 compares architectures and not tokenisation.
- **Classical branch.** Word n -grams alone were weaker than word plus character n -grams, and fitting the character vectoriser on the divergent tail of each option beat fitting it on the full text, because a prefix shared by all five options carries no discriminative signal.

2.5 Note on Ensembling

A weighted blend of logistic regression, a ridge classifier and an SGD classifier at 0.50/0.30/0.20 was built and then dropped, because it scored below logistic regression alone. Ridge returns an unbounded decision margin rather than a probability, so blended raw it dominates by scale; calibrating it through a sigmoid fixed the scale but not the outcome, since ridge was the weakest member and a weight of 0.30 on it dragged the average down.

3 Training Details & Hyperparameter Tuning

Model	Pretrained?	Optimiser	LR	Epochs	Batch	Dropout
TF-IDF + LR	No	L-BFGS	—	—	—	—
TextCNN	No (scratch)	AdamW + cosine	5e-4	25	32	0.5
BERT-base-uncased	Yes	AdamW + warmup	2e-5	4	16	0.1

Table 1: Training configuration per model. All three use the same 5-fold split and log to the same WandB project, so runs are directly comparable. TextCNN adds weight decay 0.01; BERT uses a linear schedule with 10% warmup and all layers unfrozen. Rather than patience-based early stopping, each neural model keeps the checkpoint from its best validation MAP@3 epoch.

Hyperparameter tuning notes. Key trials:

- **TextCNN regularisation.** The first implementation used a 128-dimensional embedding and no dropout, giving 4.0M parameters, and its validation MAP@3 peaked at epoch 1 and degraded for the 24 epochs after. Halving the embedding to 64 dimensions, adding the dropout of 0.5 that Kim’s paper specifies and switching from Adam to AdamW with weight decay 0.01 removed the overfitting entirely and halved the parameter count to 2.0M.
- **BERT warmup.** Two epochs at a flat 2e-5 left the randomly initialised classification head uncalibrated, with recall well below precision. Four epochs with a linear schedule and 10% warmup fixed it, since warmup matters when the head is random while the encoder is not.
- **Logistic regression, C sweep.** Inverse regularisation strength was swept across four orders of magnitude, from 0.001 to 10, and logged as a WandB sweep. Performance was flat across most of the range, so C was not a sensitive knob on this feature set.
- **Best-epoch checkpointing.** Each neural model restores the epoch with the best validation MAP@3 before generating predictions, rather than keeping the final epoch. For TextCNN this was worth 0.043 MAP@3.

4 WandB Tracking & Model Comparison

Both neural models log identical per-epoch metrics to the same WandB project — train/val loss, val/MAP@3, val/accuracy and val/F1 (macro) — so the two runs can be compared directly on the dashboard as well as here.

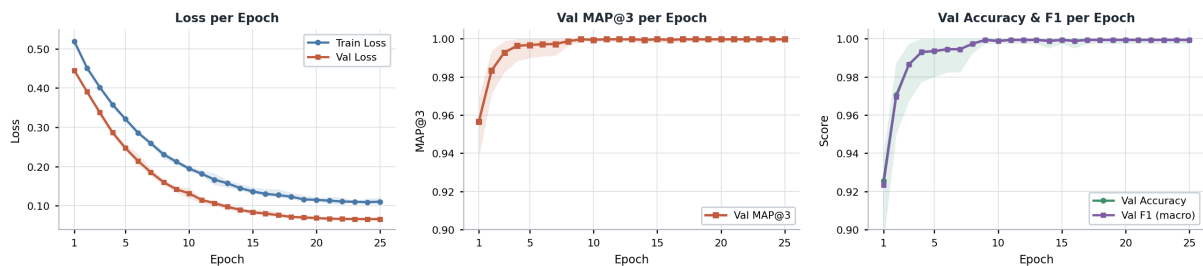


Figure 3: TextCNN — training/validation loss, validation MAP@3, and validation accuracy/F1 per epoch over 25 epochs, averaged across five folds with min-max bands.

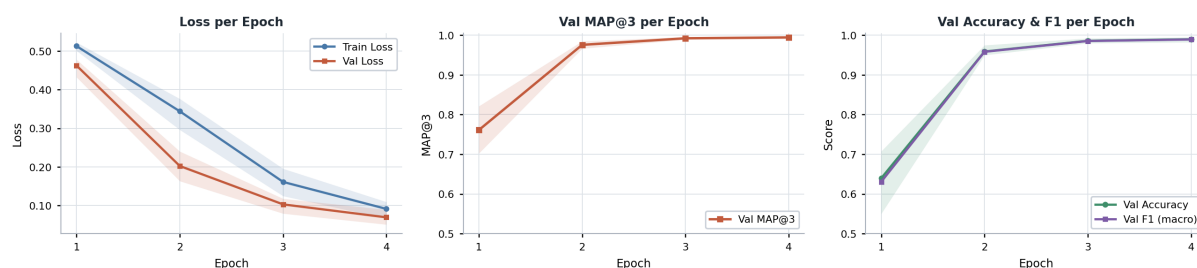


Figure 4: BERT-base-uncased — the same three panels over 4 epochs.

Model	Best Val MAP@3	Best Val Accuracy	Best Val F1 (macro)
TF-IDF + Logistic Regression	0.9974	—	—
TextCNN (from scratch)	1.0000	0.9995	0.9994
BERT-base-uncased	0.9946	0.9900	0.9894

Table 2: Best validation metrics, logged identically across the WandB runs for direct comparison.

Read these validation numbers with the duplicate-row finding in mind

All three models reach near-perfect validation scores. This is consistent with, and explained by, the duplication found in Section 1.1: when one paraphrase of a question lands in the training fold and another lands in the validation fold, a model can score that validation row correctly by memorisation rather than by genuine generalisation. Because of this, **leaderboard scores (Section 5), not these validation numbers, are treated as the trustworthy metric throughout this report** — the leaderboard’s hidden test answers are never exposed to anything trained on `train.csv`.

5 Evaluation

The submitted pipeline scored **0.75561 MAP@3** on the public leaderboard, which is calculated on approximately 80% of the test data, against 0.352 for the constant prediction B C A and 0.3667 for a uniformly random ranking. Predictions were ranked on the averaged logistic regression test probabilities.

Internal validation MAP@3 is deliberately not treated as the primary metric. All three models sit above 0.99 on validation while the leaderboard sits at 0.756, and the gap is the duplication described in Section 1.1 rather than variance. The leaderboard, which never sees training labels, is the only fully trustworthy comparison signal in this project.

6 Error Analysis

Failure patterns. Mistakes cluster around close-paraphrase distractors rather than appearing randomly across questions. In one question on interstellar absorption, three of the four distractors were the correct 207-character sentence with a single edit: “neutral” became “charged”, “hydrogen” became “helium”, and “121.5 nanometers” became “212.5 nanometers”. A model relying on surface word overlap has almost nothing to distinguish these by, which is exactly why the character vectoriser over the divergent tail carries the classical model. TextCNN can detect the changed token but has no mechanism to check it against the question.

Model agreement. All three models select the same rank-1 letter on 498 of 500 test rows, and TextCNN and BERT agree on all 500. Their full top-3 orderings, however, match on only 45.4% of rows for logistic regression and TextCNN, and 22.6% for TextCNN and BERT, so the disagreement lives in positions two and three. Three architectures spanning four orders of magnitude converging on the same first choice is why the blend in Section 2.5 had nothing to work with: there is no error diversity left to combine.

A verified negative result. 257 of the 500 test rows have an exact signature match in `train.csv`, making their true answer independently knowable without touching the hidden leaderboard labels. A post-processing step that force-ranked this known answer to position 1 whenever it disagreed with a model's own top choice changed almost nothing: TextCNN and BERT were already correct on all 257 and logistic regression on all but one. This confirms that the remaining error sits entirely in the ~243 genuinely novel test questions, and that a train/test lookup shortcut has nothing left to contribute.

7 Insights & Conclusion

- A TextCNN with no pretrained knowledge matched a 109.5M-parameter pretrained transformer on every metric tracked, suggesting this dataset rewards lexical discrimination between close options more than deep world knowledge.
- Validation metrics on this dataset cannot be trusted at face value once the duplication is accounted for; the leaderboard was the only safe comparison signal used throughout.
- Regularisation mattered far more than capacity. Halving TextCNN's embedding and adding dropout removed overfitting that no amount of extra training would have fixed, and giving BERT a warmup schedule fixed an uncalibrated head that more epochs alone did not.
- Several reasonable ideas failed for specific, diagnosable reasons rather than bad luck: an ensemble that regressed through unbounded ridge margins, and a train/test lookup override with nothing left to contribute. Confirming why took direct verification, not intuition.

Conclusion. Three models spanning 53 thousand to 109.5 million parameters were built, tracked end to end through WandB and compared on identical metric names, reaching **0.75561 MAP@3** on the public leaderboard. The most consequential finding was not a modelling change but the duplication in the data, which explains near-perfect validation scores that would otherwise have looked like success. Given a training set that small, further gains likely require more labelled data rather than continued tuning of this model family.

7.1 Future Work

- Add a recurrent model, a bidirectional LSTM or GRU over the same tokenised inputs, to isolate how much of a transformer’s advantage comes from attention specifically rather than from sequence modelling in general.
- Resolve the 44 questions that carry contradictory correct-answer text before training, either by majority-voting across their paraphrases or by dropping those rows, since they currently contribute conflicting gradients.
- Evaluate on the ~ 243 test rows with no exact match in train, which measure generalisation alone in a way the aggregate leaderboard score does not.
- Test whether better pretraining helps where more parameters did not, using DeBERTa-v3 or an entailment-pretrained cross-encoder, and a LambdaRank gradient-boosted ranker that optimises the ranking objective directly.

References: [1] Vaswani, A. et al., “Attention Is All You Need,” *NeurIPS* 30, 2017, arXiv:1706.03762. [2] Kim, Y., “Convolutional Neural Networks for Sentence Classification,” *EMNLP 2014*, pp. 1746–1751, arXiv:1408.5882. [3] PyTorch documentation, pytorch.org/docs/stable.